



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

Aplicaciones para Comunicaciones en Red

TEMA:

**Práctica: Streaming de Audio en Tiempo Real con
Go-Back-N sobre UDP**

ALUMNO:

Erreguin Franco Yair Alejandro

Boleta: 2022630121

PROFESOR:

Moreno Cervantes Axel Ernesto

26 de junio de 2026

Índice

1. Introducción	2
1.1. UDP y Go-Back-N	2
1.2. Hilos y Concurrencia	2
1.3. La Tubería (Pipe)	2
1.4. Sincronización Natural: el Modelo Productor-Consumidor	3
1.5. Streaming de Audio	3
2. Desarrollo Técnico	4
2.1. Creación del Pipe	4
2.2. Lanzamiento del Hilo de Reproducción	4
2.3. Bucle de Descarga GBN con Escritura al Pipe	5
2.4. Cierre del Pipe y Espera al Hilo de Reproducción	5
2.5. Cliente Completo	5
3. Flujo Completo del Sistema	9
4. Ejecución y Resultados	9
4.1. Ejecución	9
4.2. Resultados	10
5. Conclusiones	11
6. Referencias	12

1. Introducción

La práctica anterior estableció los fundamentos: fragmentar un archivo MP3, transferirlo con Go-Back-N sobre UDP y ensamblarlo al final para reproducirlo. El flujo era completamente secuencial: primero descargar todo, luego reproducir.

Esta práctica elimina esa espera. El objetivo es que la reproducción comience en cuanto lleguen los primeros fragmentos, descargando y reproduciendo al mismo tiempo mediante dos hilos y una tubería (*pipe*). El resultado es un sistema de **streaming en tiempo real**: el usuario escucha la canción mientras aún se está transfiriendo, sin necesidad de guardar nada en disco.

Los conceptos que hacen posible esto son: el modelo productor-consumidor, las tuberías de datos entre hilos y la sincronización natural que surge de ellas. A continuación se explica cada uno.

1.1. UDP y Go-Back-N

Estos fundamentos se mantienen igual que en la práctica anterior. UDP no garantiza entrega ni orden, y Go-Back-N implementa esa garantía de forma manual: el emisor envía una ventana de paquetes, el receptor acepta solo los que llegan en orden estricto, y ante cualquier pérdida el emisor retrocede al inicio de la ventana y reenvía todo.

Lo que cambia en esta práctica no es el protocolo de transferencia, sino lo que el cliente hace con cada fragmento en cuanto lo recibe.

1.2. Hilos y Concurrencia

Un hilo (*thread*) es una unidad de ejecución independiente dentro de un mismo programa. Dos hilos comparten la misma memoria del proceso pero avanzan de forma paralela, cada uno ejecutando su propio código.

En esta práctica el cliente utiliza dos hilos con roles bien definidos:

- **Hilo de descarga:** corre el bucle GBN, recibe datagramas UDP y escribe cada fragmento en cuanto llega en orden.
- **Hilo de reproducción:** corre el decodificador JLayer, lee datos continuamente y los convierte en audio.

Sin hilos, estas dos tareas tendrían que turnarse: descargar un poco, reproducir un poco, descargar un poco... lo cual es complejo de coordinar y produce interrupciones. Con hilos, ambas corren al mismo tiempo sin interferirse.

1.3. La Tubería (Pipe)

Una tubería es un canal de comunicación unidireccional entre dos hilos. Tiene dos extremos:

- **PipedOutputStream:** el extremo de escritura. El hilo de descarga escribe fragmentos aquí.

- **PipedInputStream:** el extremo de lectura. El hilo de reproducción lee datos desde aquí.

Lo que entra por un extremo sale por el otro, en el mismo orden, como una tubería física. El pipe tiene un buffer interno (1024 bytes por defecto en Java) que actúa como zona de amortiguación entre los dos hilos.

Hilo descarga	pipe (buffer)	Hilo reproducción
-- fragmento 0 -->	[0 1 2]	----->
-- fragmento 1 -->	[0 1 2 3]	(consume)
(bloqueado)	[1 2 3]	----->
-- fragmento 4 -->	[1 2 3 4]	

1.4. Sincronización Natural: el Modelo Productor-Consumidor

El modelo productor-consumidor describe una situación donde un hilo genera datos (*productor*) y otro los consume (*consumidor*), compartiendo un buffer intermedio. La sincronización entre ellos surge del propio comportamiento del buffer:

- Si el buffer está **lleno**, el productor se bloquea automáticamente hasta que el consumidor libere espacio.
- Si el buffer está **vacío**, el consumidor se bloquea automáticamente hasta que el productor escriba algo.

En esta práctica, el pipe de Java implementa exactamente este mecanismo sin que se tenga que escribir ningún `wait()`, `notify()` ni semáforo. La sincronización es *natural*: está integrada en el propio objeto `PipedOutputStream` / `PipedInputStream`.

Esto tiene una consecuencia importante: si JLayer decodifica más lento de lo que llegan los fragmentos, el hilo de descarga se frena automáticamente al llenarse el pipe. No es un error, es el mecanismo funcionando correctamente.

1.5. Streaming de Audio

En este contexto, *streaming* significa que el decodificador empieza a trabajar antes de tener el archivo completo. JLayer (la biblioteca de decodificación MP3 utilizada) acepta cualquier `InputStream` como fuente, no necesariamente un archivo. Al pasarle el `PipedInputStream`, JLayer lee del pipe igual que leería de un archivo, con la diferencia de que los datos llegan progresivamente en lugar de estar todos disponibles desde el inicio.

Cuando el pipe está vacío (porque la red todavía no entregó el siguiente fragmento), JLayer simplemente espera bloqueado en su `read()` interno hasta que el hilo de descarga escriba más datos. Esto es transparente para el decodificador: no sabe ni le importa si la fuente es un archivo o una red.

2. Desarrollo Técnico

Esta práctica modifica únicamente el cliente. El servidor y `Paquete.java` no cambian.

2.1. Creación del Pipe

El primer cambio respecto a la práctica anterior es reemplazar el `HashMap` por el pipe. Antes el `HashMap` acumulaba todos los fragmentos para ensamblarlos al final; ahora el pipe los transmite en tiempo real al hilo de reproducción.

```
1 // Antes:
2 // Map<Integer, byte[]> recibidos = new HashMap<>();
3
4 // Ahora: los dos extremos del pipe conectados entre s
5 PipedOutputStream pipOut = new PipedOutputStream();
6 PipedInputStream pipIn = new PipedInputStream(pipOut);
```

Listing 1: Cliente.java — Creación del pipe

Conectarlos en el constructor vincula los dos extremos: todo lo que se escriba en `pipOut` quedará disponible para leer desde `pipIn`.

2.2. Lanzamiento del Hilo de Reproducción

El hilo de reproducción se lanza **antes** de entrar al bucle de descarga. Esto es lo que permite que ambas tareas corran en paralelo.

```
1 Thread hiloReproduccion = new Thread(() -> reproducir(pipIn));
2 hiloReproduccion.start();
```

Listing 2: Cliente.java — Lanzamiento del hilo de reproducción

En cuanto se ejecuta `start()`, el hilo de reproducción arranca e intenta leer del pipe. Como todavía no hay datos, se bloquea automáticamente y espera. El hilo principal continúa hacia el bucle de descarga sin detenerse.

El método `reproducir` cambia su firma para aceptar un `InputStream` en lugar de un `File`. Como `JLayer` acepta cualquier `InputStream`, el cambio es mínimo:

```
1 // Antes:
2 // static void reproducir(File archivo) {
3 //     FileInputStream fis = new FileInputStream(archivo);
4 //     Player player = new Player(fis);
5
6 // Ahora: acepta cualquier InputStream, en este caso el pipe
7 static void reproducir(InputStream stream) {
8     try {
9         Player player = new Player(stream);
10        player.play(); // bloqueante hasta que el pipe se cierre
11    } catch (Exception e) {
12        System.err.println("Error durante la reproducci n: " + e.
13                           getMessage());
14    }
15 }
```

Listing 3: Cliente.java — Método reproducir adaptado

2.3. Bucle de Descarga GBN con Escritura al Pipe

El bucle GBN es idéntico al de la práctica anterior. El único cambio es la línea donde se procesa un fragmento en orden:

```
1 if (pkt.numSec == esperado) {
2     // Antes:
3     // recibidos.put(pkt.numSec, pkt.datos);
4
5     // Ahora: escribir directo al pipe
6     // Si el pipe est lleno, esta linea se bloquea hasta que
7     // JLayer consuma datos y libere espacio (productor-consumidor)
8     pipOut.write(pkt.datos);
9
10    Paquete ack = new Paquete(Paquete.ACK, esperado, 0, new byte[0]);
11    enviar(socket, ack, addr, PUERTO);
12    esperado++;
13 } else {
14     // Fuera de orden: descartar sin ACK (GBN sin cambios)
15 }
```

Listing 4: Cliente.java — Escritura al pipe en lugar del HashMap

Esto funciona porque GBN garantiza que los fragmentos que pasan el filtro `pkt.numSec == esperado` ya están en orden. No hay huecos, no hay necesidad de reordenar nada. Cada fragmento aceptado es exactamente el siguiente byte del archivo.

2.4. Cierre del Pipe y Espera al Hilo de Reproducción

Al terminar el bucle de descarga, en lugar del bloque de ensamblado de la práctica anterior, solo se necesitan dos líneas:

```
1 // Antes: bucle for(i=0..total) + FileOutputStream + fos.write(...)
2 // Todo eso desaparece y se reemplaza por:
3
4 pipOut.close(); // se al de fin de datos para JLayer
5 hiloReproduccion.join(); // esperar a que termine de decodificar
```

Listing 5: Cliente.java — Cierre limpio del pipe

Cerrar `pipOut` es la señal que JLayer necesita para saber que no hay más datos. Cuando `PipedInputStream` detecta que su extremo de escritura fue cerrado y el buffer está vacío, devuelve `-1` en el próximo `read()`, que es la convención estándar de Java para indicar fin de stream. JLayer interpreta esto como fin del archivo y termina `player.play()` limpiamente.

El `join()` hace que el hilo principal espere a que la reproducción termine antes de cerrar el socket y salir del programa.

2.5. Cliente Completo

```
1 import java.io.*;
2 import java.net.*;
3 import java.util.*;
4 import javazoom.jl.player.Player;
```

```

5
6 public class Cliente {
7
8     static final String SERVIDOR = "localhost";
9     static final int PUERTO = 9876;
10    static final int TIMEOUT = 2000;
11
12    public static void main(String[] args) throws Exception {
13        DatagramSocket socket = new DatagramSocket();
14        InetAddress addr = InetAddress.getByName(SERVIDOR);
15        socket.setSoTimeout(TIMEOUT);
16        Scanner sc = new Scanner(System.in);
17
18        // 1. Pedir catalogo
19        Paquete req = new Paquete(Paquete.CATALOGO, 0, 0, new byte[0]);
20        enviar(socket, req, addr, PUERTO);
21
22        byte[] buf = new byte[65535];
23        DatagramPacket dp = new DatagramPacket(buf, buf.length);
24        socket.receive(dp);
25        Paquete catalogo = Paquete.desdBytes(dp.getData(), dp.getLength())
26            ;
27
28        String[] canciones = new String(catalogo.datos).split("\n");
29        System.out.println("=== Catalogo del Servidor ===");
30        for (int i = 0; i < canciones.length; i++) {
31            System.out.println(i + ". " + canciones[i]);
32        }
33
34        // 2. El usuario elige
35        System.out.print("\nNumero de cancion a reproducir: ");
36        int eleccion = Integer.parseInt(sc.nextLine().trim());
37
38        if (eleccion < 0 || eleccion >= canciones.length) {
39            System.out.println("Seleccion invalida.");
40            socket.close();
41            return;
42        }
43
44        // 3. Pedir la cancion
45        Paquete pedir = new Paquete(Paquete.PEDIR, eleccion, 0, new byte
46            [0]);
47        enviar(socket, pedir, addr, PUERTO);
48        System.out.println("Descargando y reproduciendo '"
49            + canciones[eleccion] + "'...");
50
51        // 4. Crear el pipe
52        PipedOutputStream pipOut = new PipedOutputStream();
53        PipedInputStream pipIn = new PipedInputStream(pipOut);
54
55        // 5. Lanzar hilo de reproduccion
56        Thread hiloReproduccion = new Thread(() -> reproducir(pipIn));
57        hiloReproduccion.start();
58    }
59 }

```

```

57 // 6. Bucle de descarga GBN
58 int esperado = 0;
59
60 while (true) {
61     try {
62         byte[] rbuf = new byte[65535];
63         DatagramPacket rdp = new DatagramPacket(rbuf, rbuf.length)
64         ;
65         socket.receive(rdp);
66         Paquete pkt = Paquete.desdBytes(rdp.getData(), rdp.
67             getLength());
68
69         if (pkt.tipo == Paquete.FIN) {
70             System.out.println("\nFIN recibido. Total paquetes: "
71                 + pkt.numSec);
72             break;
73         }
74
75         if (pkt.tipo == Paquete.DATOS) {
76             System.out.print("\r Recibido seq=" + pkt.numSec
77                 + "/" + (pkt.total - 1));
78
79             if (pkt.numSec == esperado) {
80                 pipOut.write(pkt.datos);
81                 Paquete ack = new Paquete(Paquete.ACK, esperado,
82                     0, new byte[0]);
83                 enviar(socket, ack, addr, PUERTO);
84                 esperado++;
85             } else {
86                 System.out.print(" [fuera de orden, descartado]");
87             }
88         }
89     } catch (SocketTimeoutException e) {
90         System.out.println("\nTimeout esperando datos.");
91         break;
92     }
93 }
94
95 // 7. Cerrar pipe y esperar reproduccion
96 pipOut.close();
97 hiloReproduccion.join();
98
99 socket.close();
100 System.out.println("Reproduccion finalizada.");
101
102 static void reproducir(InputStream stream) {
103     try {
104         Player player = new Player(stream);
105         player.play();
106     } catch (Exception e) {
107         System.err.println("Error durante la reproduccion: "
108             + e.getMessage());
109     }
110 }

```

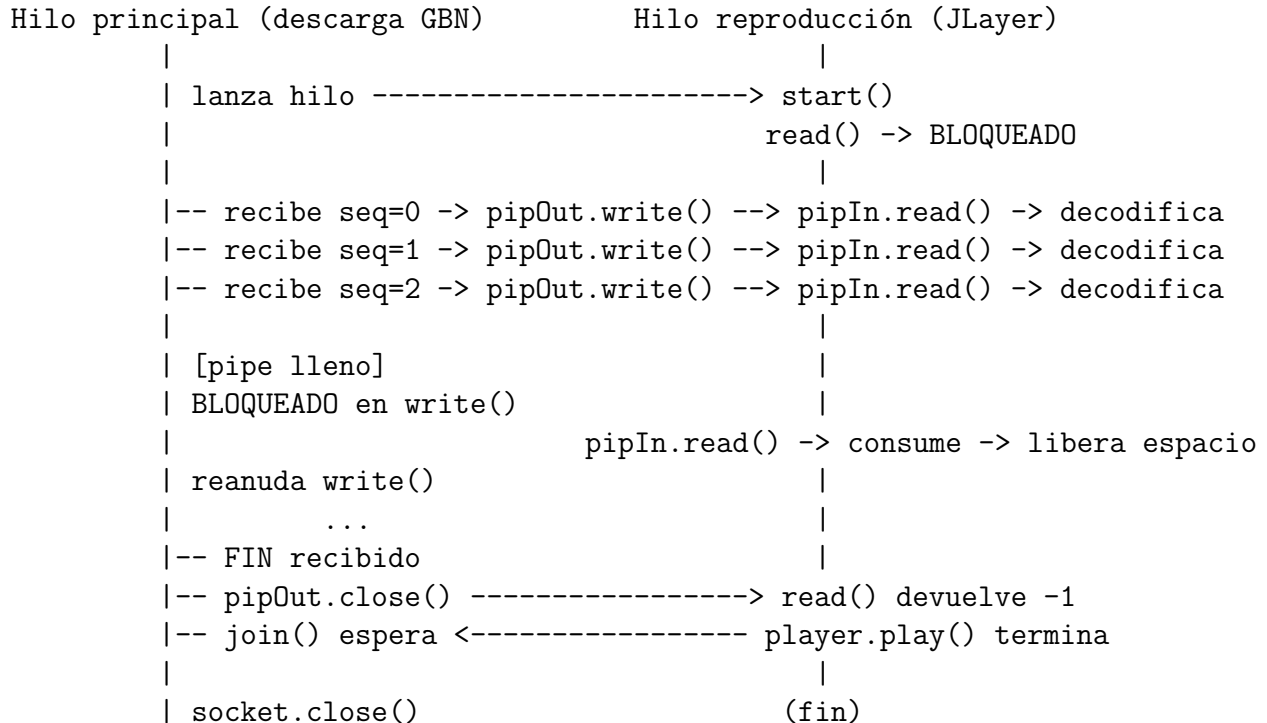
```

109     }
110
111     static void enviar(DatagramSocket s, Paquete p,
112                       InetAddress addr, int puerto) throws Exception {
113         byte[] raw = p.aBytes();
114         s.send(new DatagramPacket(raw, raw.length, addr, puerto));
115     }
116 }

```

Listing 6: Cliente.java — Versión completa con streaming

3. Flujo Completo del Sistema

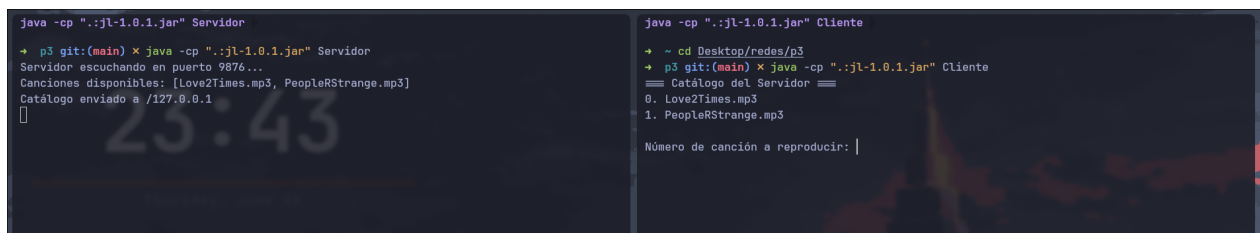


El diagrama muestra los dos puntos clave de sincronización natural: el hilo de reproducción bloqueado esperando datos al inicio, y el hilo de descarga bloqueado cuando el pipe se llena. Ninguno de los dos bloqueos requiere código explícito de sincronización.

4. Ejecución y Resultados

4.1. Ejecución

Para llevar a cabo la ejecución se creo un shell script el cual automatiza el compilado y ejecución tanto del cliente como del servidor. A continuación se muestra el resultado de la ejecución:



```
java -cp ".:j1-1.0.1.jar" Servidor
+ p3 git:(main) x java -cp ".:j1-1.0.1.jar" Servidor
Servidor escuchando en puerto 9876...
Canciones disponibles: [Love2Times.mp3, PeopleRStrange.mp3]
Catálogo enviado a /127.0.0.1
25:43

java -cp ".:j1-1.0.1.jar" Cliente
+ ~ cd Desktop/redes/p3
+ p3 git:(main) x java -cp ".:j1-1.0.1.jar" Cliente
== Catálogo del Servidor ==
0. Love2Times.mp3
1. PeopleRStrange.mp3
Número de canción a reproducir: |
```

Figura 1: Log de la consola del servidor y del cliente conectado.

4.2. Resultados

En el catálogo de esta práctica se agregaron dos canciones, a diferencia de la práctica anterior, aquí ya no hay un menú interactivo, al seleccionar una canción directamente se empieza a descargar y al mismo tiempo a reproducir, es decir, existe un streaming udp entre el cliente y el servidor. A continuación se muestra dicho streaming de la canción:



```
java -cp ".:jl-1.0.1.jar" Servidor
ACK=238
TX seq=242
ACK=239
TX seq=243
ACK=240
TX seq=244
ACK=241
TX seq=245
ACK=242
TX seq=246
ACK=243
TX seq=247
ACK=244
TX seq=248
ACK=245
TX seq=249
ACK=246
TX seq=250
ACK=247
TX seq=251
ACK=248
TX seq=252
ACK=249
TX seq=253
ACK=250
TX seq=254
ACK=251
TX seq=255
ACK=252
TX seq=256
ACK=253
TX seq=257
ACK=254
TX seq=258
ACK=255
TX seq=259
ACK=256
TX seq=260

java -cp ".:jl-1.0.1.jar" Cliente
+ ~ cd Desktop/redes/p3
+ p3 git:(main) x java -cp ".:jl-1.0.1.jar" Cliente
== Catálogo del Servidor ==
0. Love2Times.mp3
1. PeopleRStrange.mp3

Número de canción a reproducir: 0
Descargando y reproduciendo 'Love2Times.mp3'...
Recibido seq=257/4607
```

Figura 2: Streaming en tiempo real entre el cliente y el servidor.

Al terminar el streaming, se cierra el pipe creado entre hilos, se finalizan ambos hilos y el cliente se detiene. Si un cliente vuelve a conectarse entonces se puede volver a hacer el streaming udp:



```
java -cp ".:jL-1.0.1.jar" Servidor
TX seq=4591
ACK=4588
TX seq=4592
ACK=4589
TX seq=4593
ACK=4590
TX seq=4594
ACK=4591
TX seq=4595
ACK=4592
TX seq=4596
ACK=4593
TX seq=4597
ACK=4594
TX seq=4598
ACK=4595
TX seq=4599
ACK=4596
TX seq=4600
ACK=4597
TX seq=4601
ACK=4598
TX seq=4602
ACK=4599
TX seq=4603
ACK=4600
TX seq=4604
ACK=4601
TX seq=4605
ACK=4602
TX seq=4606
ACK=4603
TX seq=4607
ACK=4604
ACK=4605
ACK=4606
ACK=4607
FIN enviado.
[]

~/Desktop/redes/p3
+ ~ cd Desktop/redes/p3
+ p3 git:(main) x java -cp ".:jL-1.0.1.jar" Cliente
== Catálogo del Servidor ==
0. Love2Times.mp3
1. PeopleRStrange.mp3

Número de canción a reproducir: 0
Descargando y reproduciendo 'Love2Times.mp3'...
Recibido seq=4607/4607
FIN recibido. Total paquetes: 4608
Reproducción finalizada.
+ p3 git:(main) x |
```

Figura 3: Streaming finalizado.

5. Conclusiones

La diferencia entre esta práctica y la anterior es conceptualmente pequeña en código (unas quince líneas cambiadas) pero grande en lo que representa: pasar de un modelo secuencial a uno concurrente.

El modelo productor-consumidor con pipe demostró ser una solución limpia para este problema. La sincronización entre hilos no requirió ni un semáforo ni un `synchronized`: el propio pipe regula el ritmo entre quien produce datos y quien los consume. Cuando el buffer se llena, el productor espera; cuando se vacía, el consumidor espera. El sistema se autoequilibra.

El hecho de que JLayer acepte cualquier `InputStream` fue clave para que el cambio fuera tan acotado. El decodificador no sabe si está leyendo un archivo o una red; para él es indiferente. Esa abstracción de `InputStream` en Java es la que hace posible conectar componentes de orígenes distintos sin modificarlos.

Finalmente, vale la pena notar que lo que se implementó aquí, aunque a pequeña escala, es el principio detrás de cualquier servicio de streaming real: no esperar a tener el contenido completo para empezar a procesarlo, sino trabajar con los datos conforme llegan.

6. Referencias

Referencias

- [1] A. S. Tanenbaum y D. Wetherall, *Computer Networks*, 5th ed. Prentice Hall, 2011.
- [2] J. F. Kurose y K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Pearson, 2017.
- [3] Oracle, “PipedInputStream (Java SE docs),” 2025. [En línea]. Disponible: <https://docs.oracle.com/en/java/javase/>. [Accedido: 25-jun-2026].
- [4] Oracle, “Thread (Java SE docs),” 2025. [En línea]. Disponible: <https://docs.oracle.com/en/java/javase/>. [Accedido: 25-jun-2026].
- [5] J. Postel, “RFC 768 — User Datagram Protocol,” IETF, 1980. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc768>. [Accedido: 25-jun-2026].